CS 3006
Parallel and Distributed Computing
Lecture 10 + 11

Danyal Farhat

FAST School of Computing

NUCES Lahore

Shared Memory Programming with OpenMP

Lecture's Agenda

- **Shared Memory Programming**
- **Shared-Memory Programming (Fork/Join Parallelism)**
- **Shared-Memory vs. Message Passing Model**
- **Introduction to OpenMP**
- **OpenMP Solution Stack Model**
- **Implementation using Visual Studio C++**
- **Shared and Private Variables**
- **Basic Library Functions**

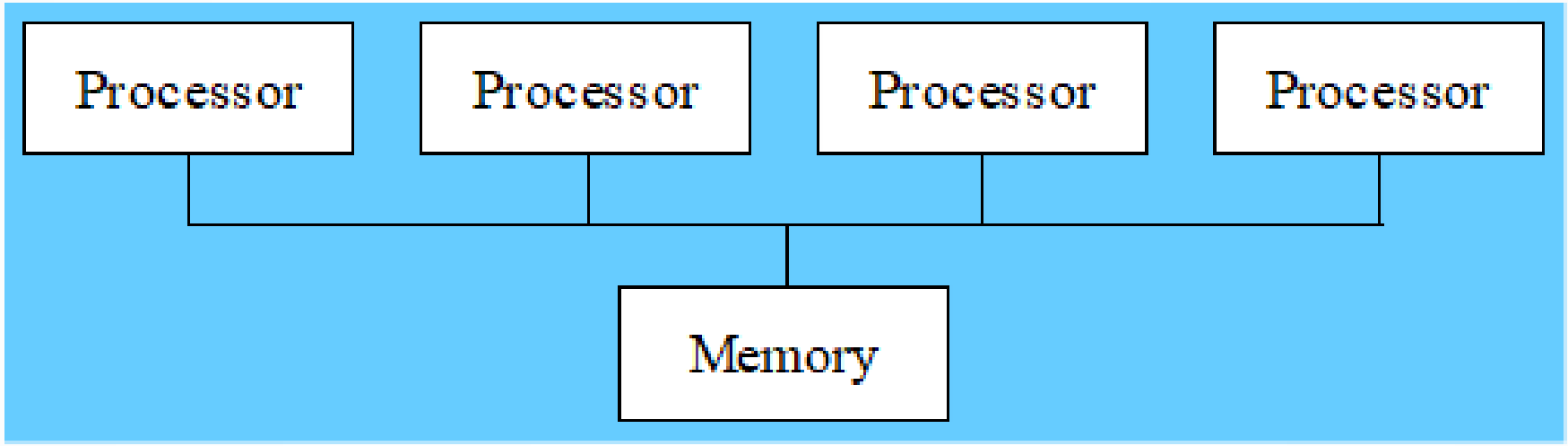
Lecture’s Agenda (Cont.)

- **Program Structure**
- **First Program: Hello World**
- **Summary**
- **Additional Resources**

Shared Memory Programming

- **Physically: processors in a computer share access to the same RAM**
- **Virtually: threads running on the processors interact with one another via shared variables in the common address space of a single process**
- **Making performance improvements to serial code tends to be easier with multithreading than with message passing paradigm**
  - **Message passing usually requires the code/algorithm to be redesigned**
  - **Multithreading allows incremental parallelism: take it one loop at a time**
- **Clusters today are commonly made up of multiple processors per compute node; using OpenMP with MPI is a strategy to improve performance at the two levels (i.e. shared and distributed memory)**

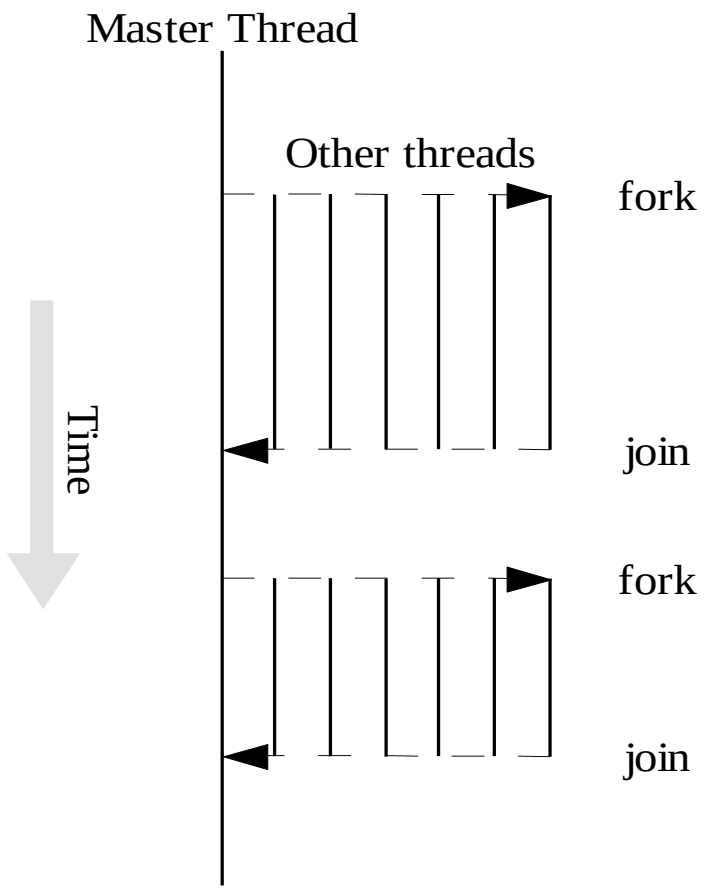
Shared Memory Programming (Cont.)



Processors interact and synchronize with each other through shared variables.

Shared-Memory Programming (Fork/Join Parallelism)

- **Multi-threaded programming is the most common shared-memory programming methodology**
- **A serial code begins execution. The process is the master thread or only executing thread**
- **When a parallel portion of the code is reached, the Master thread can "fork" more threads to work on it**
- **When the parallel portion of the code has completed, the threads "join" again, and the master thread continues executing the serial code**



Shared-Memory Model vs. Message Passing Model

Shared-Memory Model

- One active thread at start and end of the program
- Number of active threads inside program changes dynamically during execution
- Supports incremental parallelism
  - The process of converting a sequential program to a parallel program a little bit at a time

Message-Passing Model

- All processes remain active throughout execution of program
- Sequential-to-parallel transformation requires major effort
- No incremental parallelism
  - Transformation done in one giant step rather than many tiny steps

Introduction to OpenMP

- **OpenMP (Open Multi Processing) is an API for writing multithreaded applications**
- **Provides an implementation model to distribute and decompose the work across multiple processors**
- **Uses threads to deploy work**
- **Greatly simplifies writing multi-threaded (MT) programs in Fortran, C and C++**

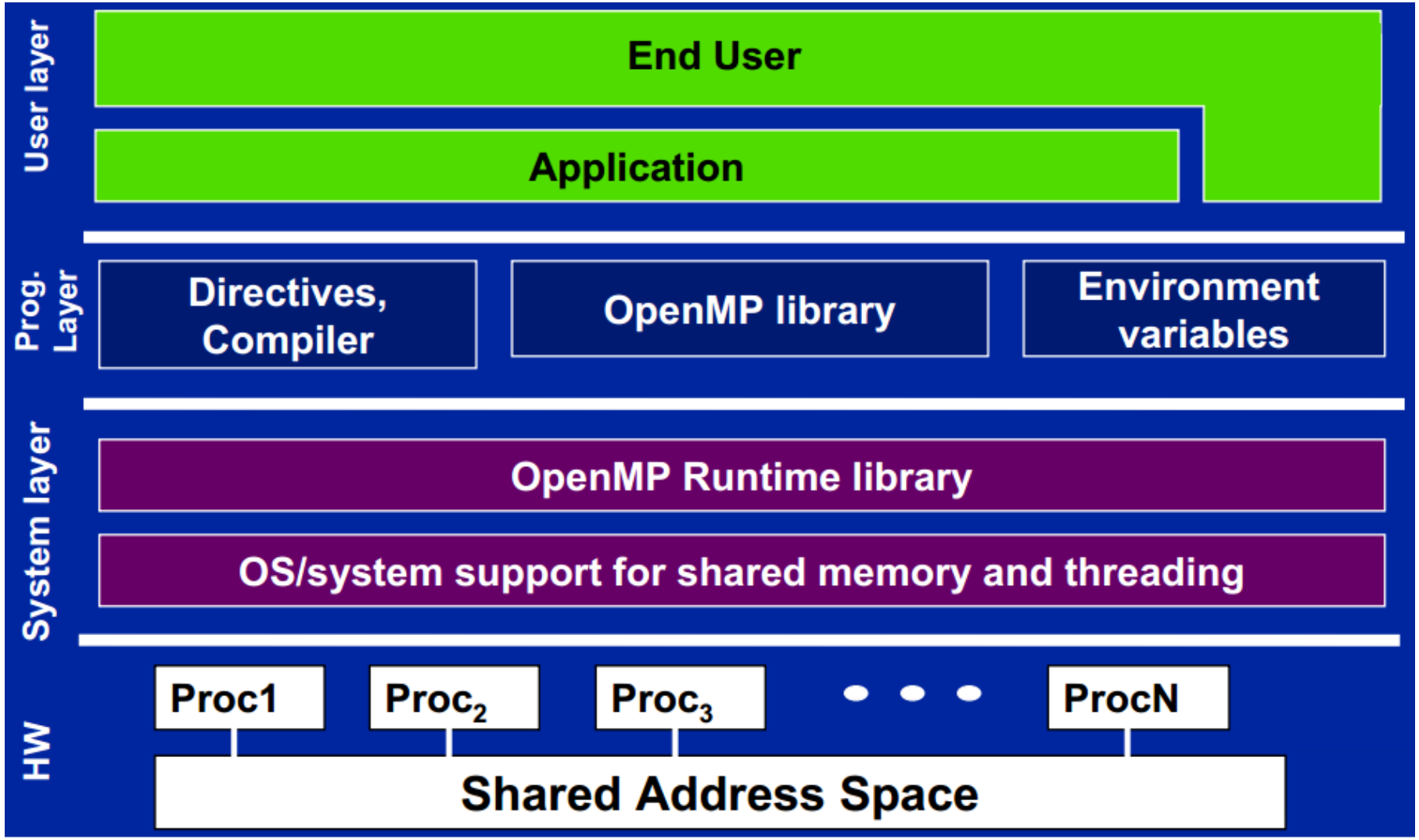
Introduction to OpenMP (Cont.)

- **OpenMP is described by the API based on:**
  - **A set of compiler directives for depicting the parallelism in the source code**
  - **A library of subroutines**
  - **A set of Environment Variables**

- **OpenMP directives in C and C++ are based on the #pragma compiler directives**

- **Works on the basis of Fork-Join model**

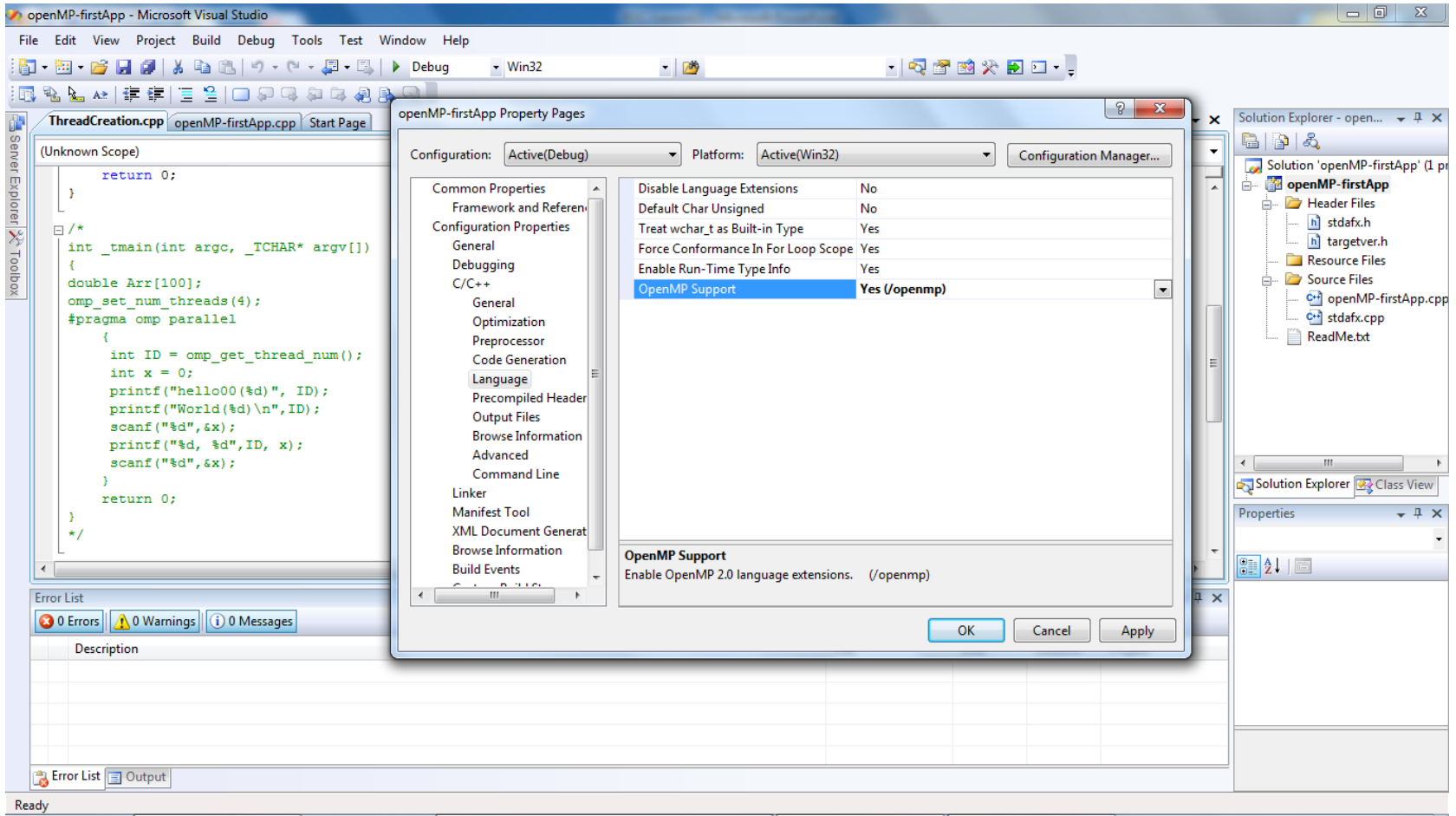
OpenMP Solution Stack Model



Source: http://openmp.org/mp-documents/omp-hands-on-SC08.pdf

Implementation using Visual Studio C++

- **Turn on OpneMP support in Visual Studio**
- **Project properties → Configuration → C/C++/Language**



Shared and Private Variables

Shared Variable:

- **Has same address in execution context of every thread**

Private Variable:

- **Has different address in execution context of every thread**

- **A thread cannot access the private variables of another thread**

Basic Library Functions

- **omp_get_num_procs**

```
int procs = omp_get_num_procs()
                                        //number of CPUs/cores in machine
```

- **omp_get_num_threads**

```
int threads = omp_get_num_threads()
                                        //number of active threads
                                        //should be called from a parallel region
```

- **omp_get_max_threads**

```
printf("Only %d threads can be forked\n",omp_get_max_threads());
                                        //can be called outside of a parallel region
```

Basic Library Functions (Cont.)

- **omp_get_thread_num**

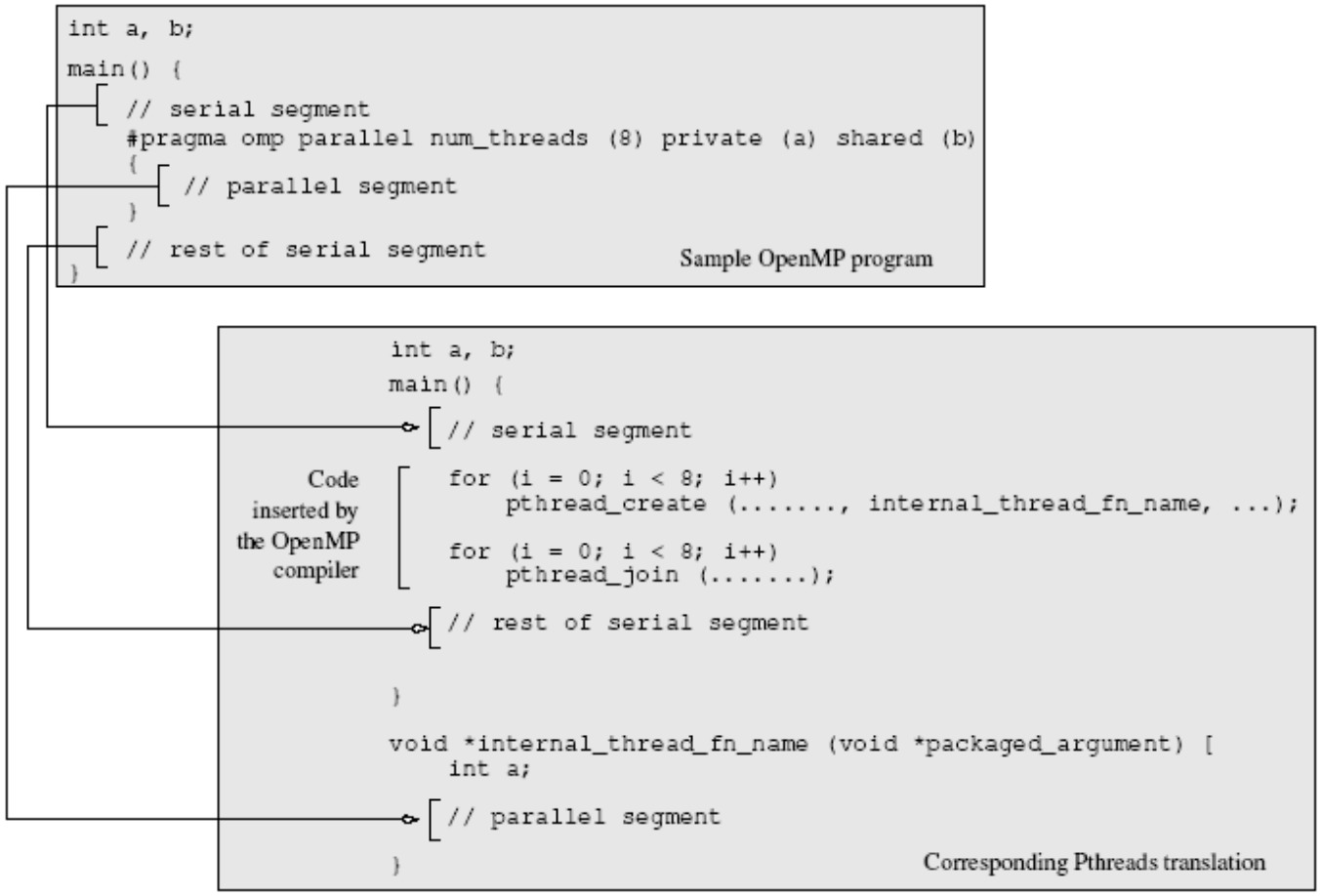
```
printf("Hello from thread id %d\n",omp_get_thread_num());
                                        // return thread id
```

- **omp_set_num_threads**

```
omp_set_dynamic(0);             // disable dynamic adjustment

omp_set_num_threads(4);         //setting thread count to 4
```

Program Structure



Source: Introduction to Parallel Computing (Karypis and Co.)

First Program: Hello World

```
#include <omp.h>
#include <iostream>
using namespace std;
int main()
{
 omp_set_num_threads(4);
 #pragma omp parallel
  {
    int Id = omp_get_thread_num();

    Printf (“hello(%d)”, Id);
    printf (“world(%d)\n”, Id)
  }
}
```

Runtime function to request a certain number of threads

Runtime function returning a thread ID

```
hello (1) hello (0) world(1)
world(0)
hello(3) hello(2) world(3)
world(2)
```

First Program: Hello World – 2nd Example

```
#include <omp.h>
int numT;
int main()
{
 #pragma omp parallel num_threads(4)
  {
    int Id = omp_get_thread_num();
    numT = omp_get_num_threads();
    Printf (“hello(%d)”, Id);
    printf (“world(%d)\n”, Id)
  }
}
```

Clause to request a certain number of threads

Runtime function returning no. of threads actually created

Summary

- **Shared Memory Programming**
  - **Processors share RAM, Threads share variables**
  - **Performance improvement is easier with multithreading than with message passing paradigm**
- **Shared-Memory Programming (Fork/Join Parallelism)**
  - **Using "fork", master thread creates worker threads**
  - **Using "join", worker threads return to the master thread**
- **Shared-Memory vs. Message Passing Model**
  - **Shared-Memory Model: One active thread at start and end of program, incremental parallelism**
  - **Message-Passing Model: All processors remain active throughout execution of program, no incremental parallelism**

Summary (Cont.)

- **Introduction to Open Multi Processing (OpenMP)**
  - **OpenMP is an API for writing multithreaded applications**
  - **OpenMP is described by the API based on:**
    - ✓ **A set of compiler directives for depicting the parallelism in the source code**
    - ✓ **A library of subroutines**
    - ✓ **A set of Environment Variables**
- **OpenMP Solution Stack Model**
- **Implementation using Visual Studio C++**
- **Shared and Private Variables**
  - **Shared Variable: Has same address in execution context of every thread**
  - **Private Variable: Has different address in execution context of every thread**

Summary (Cont.)

- **Basic Library Functions**
  - **omp_get_num_procs**
  - **omp_get_num_threads**
  - **omp_get_max_threads**
  - **omp_get_thread_num**
  - **omp_set_num_threads**
- **Program Structure**
- **First Program: Hello World**

Additional Resources

- **Introduction to Parallel Computing by Ananth Grama and Anshul Gupta**
  - **Chapter 7: Programming Shared Address Space Platforms**
- **Introduction to OpenMP by Tim Mattson (Intel)**
  - **https://www.youtube.com/watch?v=nE-xN4Bf8XI&list=PLLX-Q6B8xqZ8n8bwjGdzBJ25X2utwnoEG&index=1**

Questions?

Class Activity